

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

COMPUTER ARCHITECTUUR

Labo 4

Tano Pannekoucke (r0955339)

Academic year 2025–2026

Inhoudsopgave

1 Inleiding **2**

2 Deel 1: Coalesced Memory Access **2**
2.1 Methodologie 2
2.2 Visuele Validatie 2
2.3 Resultaten en Discussie 2

3 Deel 2: Matrix Vermenigvuldiging en Geheugentypes **4**
3.1 Implementatie van Geheugentypes 4
3.2 Resultaten: De Shared Memory Winst 4
3.3 Resultaten: De Constant Memory Anomalie 4

4 Deel 3: Bandbreedte Analyse **5**

5 Conclusie **6**

A Code Listings **7**
A.1 Deel 1: Image Processing (Coalescing) 7
A.2 Deel 2: Matrix Multiplication (Memory Types) 9
A.3 Deel 3: Bandwidth Measurement 13

1 Inleiding

Dit labo richt zich op het optimaliseren van GPU-applicaties door efficiënt gebruik van de geheugenhiërarchie. De GPU beschikt over verschillende soorten geheugen (Global, Shared, Constant), elk met specifieke eigenschappen qua latency, bandbreedte en toegangsregels. Het doel van dit verslag is om de impact van *coalesced memory access* en *shared memory tiling* te kwantificeren en te verklaren.

We analyseren twee specifieke cases:

1. Image Processing: Het verschil tussen *interleaved* (uncoalesced) en *planar* (coalesced) data-opslag.
2. Matrix Vermenigvuldiging: De prestatiewinst door het gebruik van Shared Memory en de valkuilen van Constant Memory.

2 Deel 1: Coalesced Memory Access

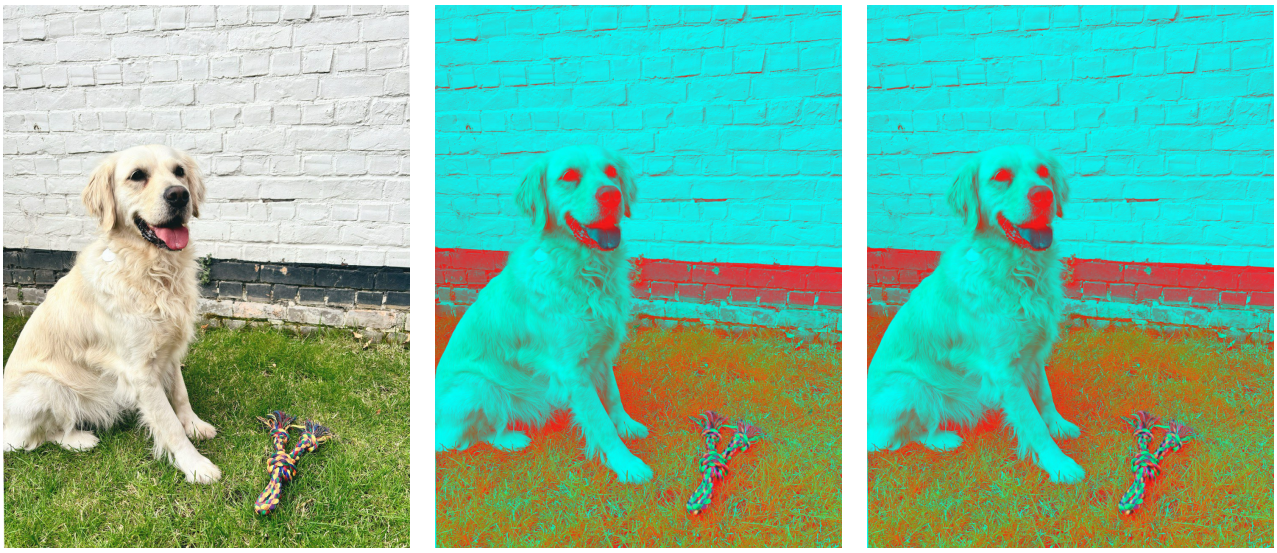
2.1 Methodologie

In dit onderdeel wordt de rode kleurcomponent van een afbeelding geïnverteerd ($R = 255 - R$). We vergelijken twee opslagformaten:

- **Uncoalesced (Array of Structures):** De pixels zijn opgeslagen als RGBRGB... Threads lezen data met een *stride* van 3 bytes. Dit verbreekt de coalescing-eisen van de hardware [1].
- **Coalesced (Structure of Arrays):** De data wordt eerst gesplitst in drie losse arrays (RRR... GGG... BBB...). Threads lezen nu sequentiële adressen, wat de hardware toestaat om de geheugentoegang samen te voegen tot één transactie.

2.2 Visuele Validatie

Om de correctheid van de kernels te verifiëren, is de output gevisualiseerd. Zoals te zien in Figuur 1, resulteert het inverteren van het rode kanaal in een cyaan-kleurige gloed (omdat Rood en Cyaan complementair zijn). Beide methoden leveren visueel identieke, correcte resultaten.

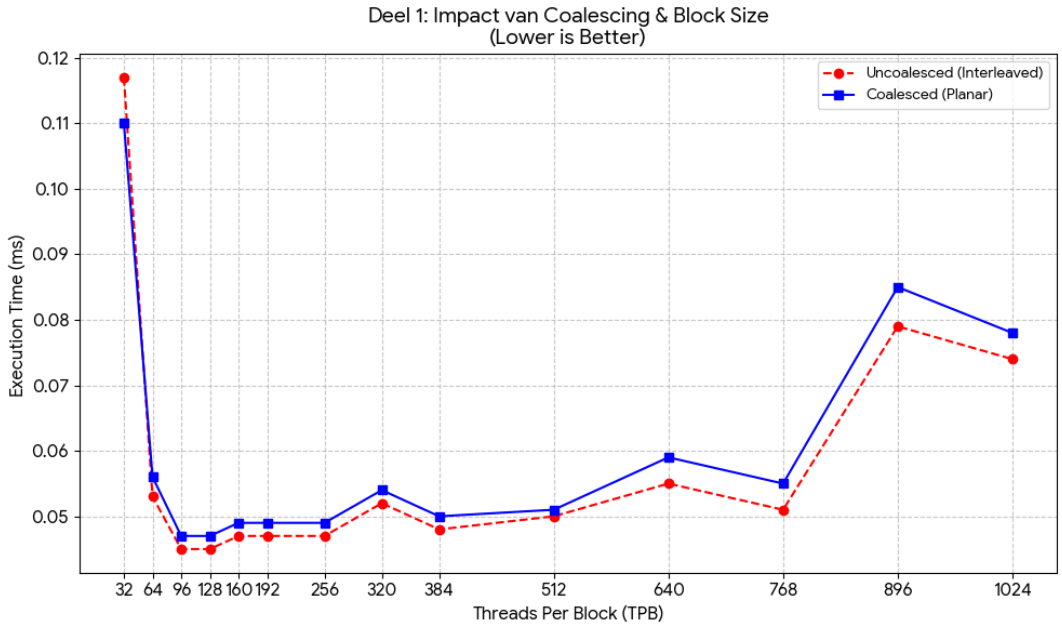


(a) Origineel (b) Uncoalesced Output (c) Coalesced Output

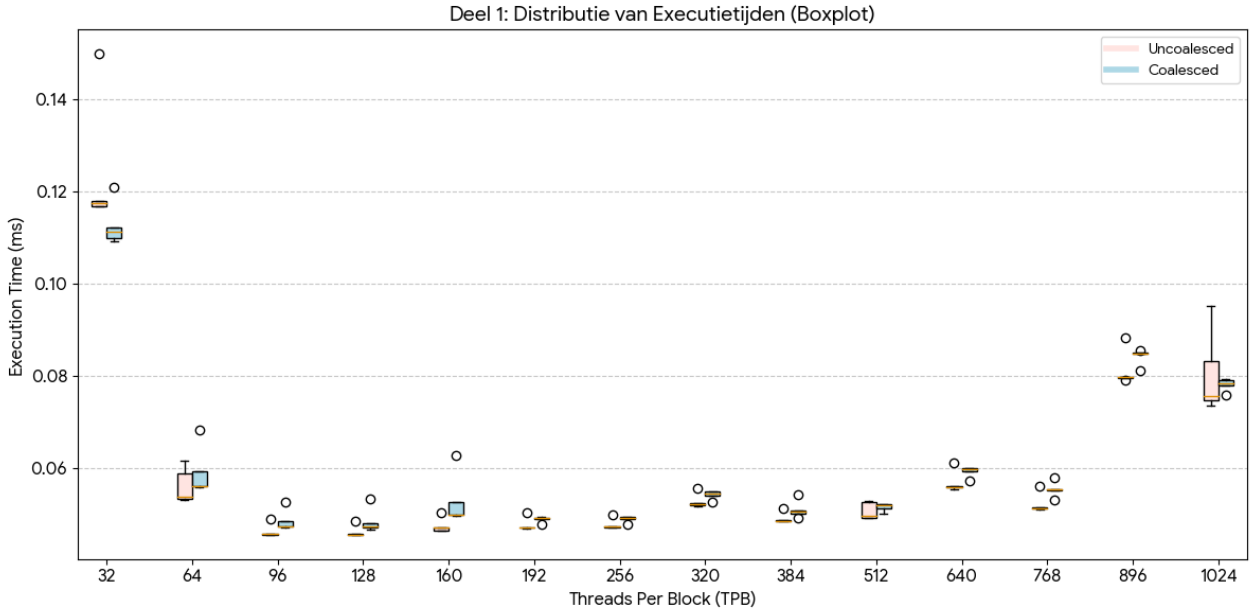
Figuur 1: Visuele inspectie van de resultaten. De rode tinten zijn correct geïnverteerd naar cyaan.

2.3 Resultaten en Discussie

Figuur 2 toont het verloop van de executietijd voor verschillende block sizes. Daarnaast toont Figuur 3 de spreiding van de metingen over meerdere runs om de stabiliteit te verifiëren.



Figuur 2: Executietijd: Uncoalesced (Rood) vs Coalesced (Blauw).



Figuur 3: Boxplot van de executietijden over 5 runs. De lage variantie bevestigt de betrouwbaarheid van de data.

Observatie: De resultaten zijn opmerkelijk. In het bereik van 96 tot 384 threads per block presteert de *uncoalesced* versie (rood) marginaal sneller dan de *coalesced* versie (blauw), wat zichtbaar is doordat de rode lijn onder de blauwe lijn duikt.

Verklaring: Dit resultaat spreekt de klassieke theorie tegen, maar is verklaarbaar door de architectuur van moderne GPU's:

1. **L2 Caching:** De stride van 3 bytes bij de interleaved opslag is klein genoeg. Wanneer een warp data ophaalt, vallen de benodigde bytes voor meerdere threads vaak binnen dezelfde cache-line. De L2-cache vangt hierdoor de inefficiëntie op, waardoor de latency-penalty minimaal is.

2. **Instructie Overhead:** De coalesced kernel vereist het beheer van drie losse pointers (voor R, G en B arrays), wat leidt tot extra registergebruik en complexere adresberekeningen per thread. De interleaved kernel heeft slechts één pointer. Omdat de rekenkern (inversie) zo licht is, weegt deze extra overhead bij de coalesced-versie zwaarder dan de winst in geheugentoegang.

3 Deel 2: Matrix Vermenigvuldiging en Geheugentypes

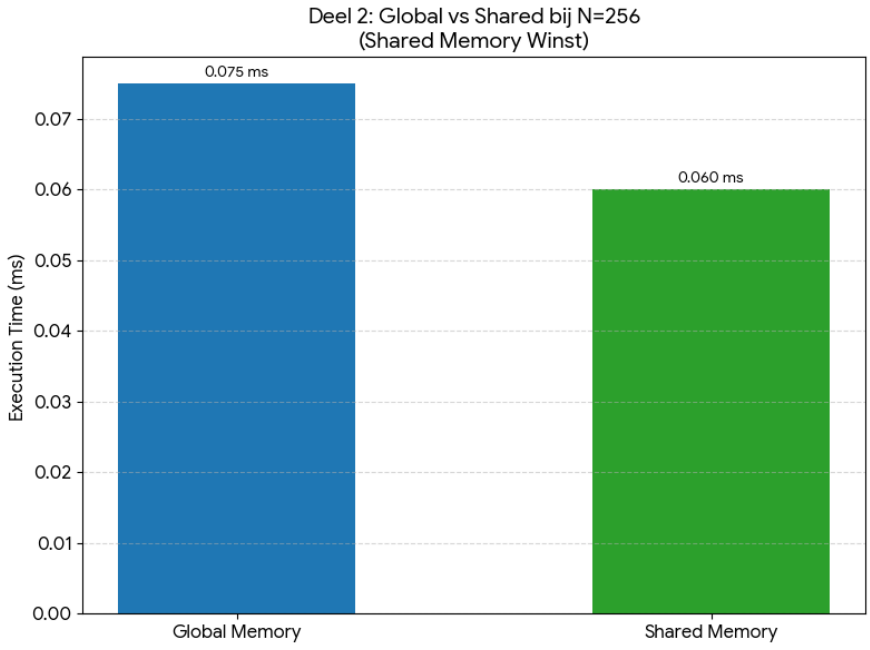
3.1 Implementatie van Geheugentypes

Voor de berekening $C = A \times B$ zijn drie kernels geïmplementeerd:

1. **Global Memory Only:** Elke thread leest direct uit het trage off-chip geheugen.
2. **Shared Memory (Tiling):** Threads laden gezamenlijk een blokje (16×16) data in het snelle on-chip shared memory [1]. Dit vermindert het aantal trips naar het globale geheugen drastisch.
3. **Constant Memory:** Matrix B wordt in het constant memory geplaatst. Dit is geoptimaliseerd voor broadcasting [1].

3.2 Resultaten: De Shared Memory Winst

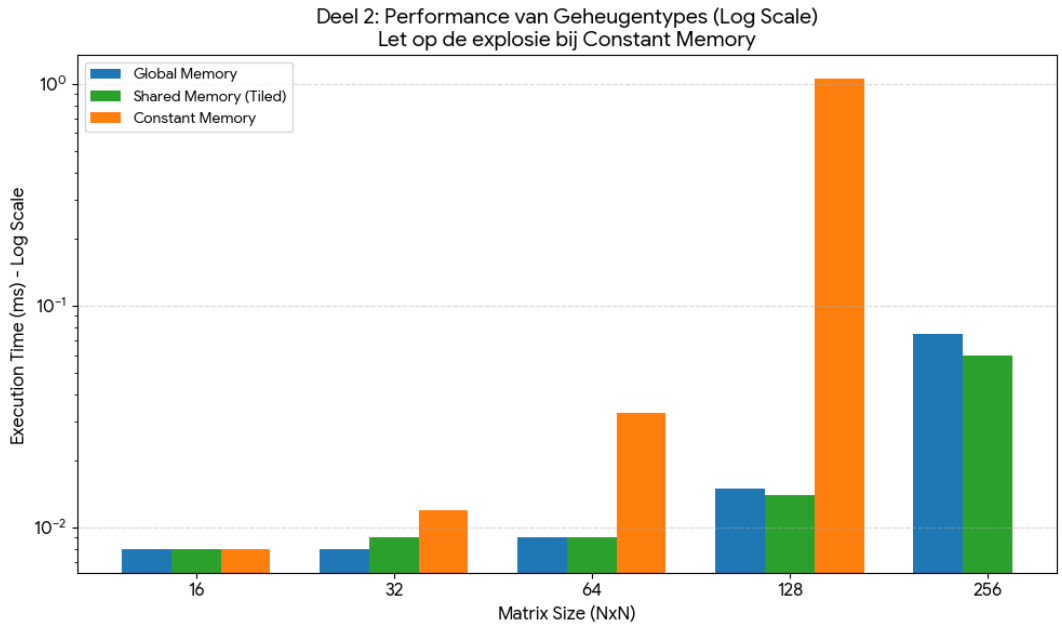
Bij grotere matrixen ($N = 256$) wordt het voordeel van Shared Memory duidelijk zichtbaar (Figuur 4). De Shared Memory kernel is hier aanzienlijk sneller omdat data lokaal hergebruikt wordt, waardoor de beperkte globale bandbreedte gespaard blijft.



Figuur 4: Executietijd bij $N = 256$. Shared Memory overtreft Global Memory.

3.3 Resultaten: De Constant Memory Anomalie

Een opvallend resultaat is de prestatie van het Constant Memory bij $N = 128$. In plaats van een versnelling, zien we een extreme vertraging (Figuur 5).

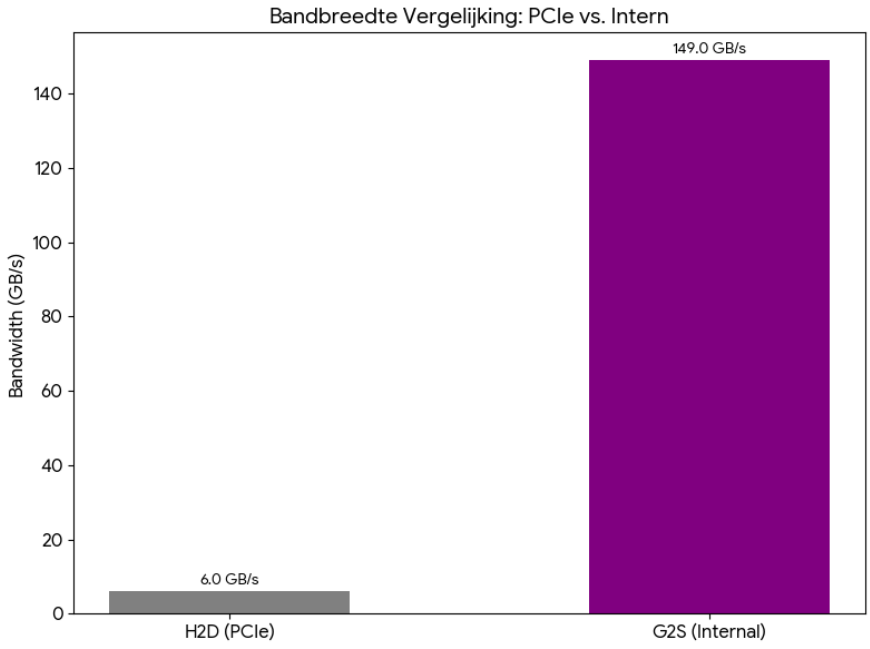


Figuur 5: Executietijden op logaritmische schaal. Let op de enorme piek bij Constant Memory voor $N = 128$.

Verklaring: Constant memory is ontworpen voor situaties waar alle threads **hetzelfde** adres lezen. Bij matrixvermenigvuldiging leest elke thread een waarde uit een **andere** kolom. Dit heet *divergent access*. De hardware moet deze reads **serialiseren**, wat leidt tot de enorme vertraging die zichtbaar is in de grafiek.

4 Deel 3: Bandbreedte Analyse

Om de prestatiewinst van Shared Memory beter te begrijpen, is de bandbreedte gemeten van CPU → GPU (H2D) en intern op de GPU (Global → Shared).



Figuur 6: Bandbreedte vergelijking: PCIe bus versus interne GPU memory bus.

Uit Figuur 6 blijkt dat de interne bandbreedte ($\approx$ 149.000 GB/s) vele malen hoger is dan de PCIe-

bandbreedte ($\approx 6.000\,\text{GB/s}$). Dit bevestigt waarom *tiling* werkt: we minimaliseren dataverkeer over de trage bus en maximaliseren gebruik van de snelle interne paden.

5 Conclusie

1. **Coalescing:** Hoewel caches de impact verkleinen, blijft Structure of Arrays (Planar) de meest efficiënte methode.
2. **Shared Memory:** Essentieel voor algoritmes met veel data-hergebruik om de "memory wall"te doorbreken.
3. **Constant Memory:** Is enkel nuttig voor uniforme toegang; divergent gebruik leidt tot desastreuze performance.

Referenties

[1] Feys, T. (2024). *Geavanceerde Computerarchitectuur Session 4: Optimization techniques, Coalesced memory access and device (GPU) memory types*. KU Leuven.

[2] NVIDIA Corporation. (2024). *CUDA C++ Programming Guide*.

[3] Google. (2025). *Gemini*. `https://gemini.google.com`

A Code Listings

A.1 Deel 1: Image Processing (Coalescing)

```cpp
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <cuda_runtime.h>

// ---- Afbeelding-dimensies ----
#define M 960
#define N 1280
#define C 3
#define OFFSET 16

// ---- CUDA util ----
#define CUDA_CHECK(call) do {                                          \
  cudaError_t _cuda_err__ = (call);                                    \
  if (_cuda_err__ != cudaSuccess) {                                    \
    fprintf(stderr, "CUDA ERROR %s:%d: %s\n", __FILE__, __LINE__,      \
            cudaGetErrorString(_cuda_err__));                          \
    std::exit(EXIT_FAILURE);                                           \
  }                                                                    \
} while(0)

// ---- I/O helpers ----
static uint8_t* get_image_array(){
    FILE* f = fopen("./anna.ppm","rb");
    if(!f){ perror("open anna.ppm"); std::exit(1); }
    size_t bytes = (size_t)M*N*C + OFFSET;
    uint8_t* buf = (uint8_t*)std::malloc(bytes);
    if(!buf){ perror("malloc"); std::exit(1); }
    size_t got = std::fread(buf,1,bytes,f);
    if(got < OFFSET){ fprintf(stderr,"ERROR: unexpected file size\n"); std::exit(1); }
    fclose(f);
    return buf + OFFSET;
}

static void save_interleaved_as_ppm(const char* path, const uint8_t* interleaved){
    FILE* f = fopen(path,"wb");
    if(!f){ perror("open out"); std::exit(1); }
    std::fprintf(f,"P6\n%d %d\n255\n", M, N);
    std::fwrite(interleaved,1,(size_t)M*N*C,f);
    fclose(f);
}

// ---- Kernels ----

// Uncoalesced (interleaved)
__global__ void invert_red_interleaved(const uint8_t* __restrict__ in,
                                       uint8_t* __restrict__ out,
                                       int pixels){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for(int p = tid; p < pixels; p += stride){
        int base = 3*p;
        out[base+0] = 255u - in[base+0];
        out[base+1] = in[base+1];
        out[base+2] = in[base+2];
    }
}

// De/interleave helpers
__global__ void deinterleave(const uint8_t* __restrict__ in,
                             uint8_t* __restrict__ R,
                             uint8_t* __restrict__ G,
                             uint8_t* __restrict__ B,
                             int pixels){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for(int p = tid; p < pixels; p += stride){
```

```
        int base = 3*p;
        R[p] = in[base+0];
        G[p] = in[base+1];
        B[p] = in[base+2];
    }
}
__global__ void interleave(const uint8_t* __restrict__ R,
                           const uint8_t* __restrict__ G,
                           const uint8_t* __restrict__ B,
                           uint8_t* __restrict__ out,
                           int pixels){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for(int p = tid; p < pixels; p += stride){
        int base = 3*p;
        out[base+0] = R[p];
        out[base+1] = G[p];
        out[base+2] = B[p];
    }
}

// Coalesced (planar)
__global__ void invert_red_planar(const uint8_t* __restrict__ Rin,
                                  uint8_t* __restrict__ Rout,
                                  const uint8_t* __restrict__ Gin,
                                  uint8_t* __restrict__ Gout,
                                  const uint8_t* __restrict__ Bin,
                                  uint8_t* __restrict__ Bout,
                                  int pixels){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for(int i = tid; i < pixels; i += stride){
        Rout[i] = 255u - Rin[i];
        Gout[i] = Gin[i];
        Bout[i] = Bin[i];
    }
}

// ---- Main ----
int main(){
    const int pixels = M * N;
    const int total  = pixels * C;

    uint8_t* hIn = get_image_array();
    uint8_t* hOutUnc = (uint8_t*)std::malloc(total);
    uint8_t* hOutCoa = (uint8_t*)std::malloc(total);
    if(!hOutUnc || !hOutCoa){ std::cerr<<"host malloc failed\n"; return 1; }

    uint8_t *dIn=nullptr,*dOut=nullptr;
    CUDA_CHECK(cudaMalloc(&dIn,total));
    CUDA_CHECK(cudaMalloc(&dOut,total));
    CUDA_CHECK(cudaMemcpy(dIn,hIn,total,cudaMemcpyHostToDevice));

    // Meer block sizes + meer iteraties
    int tpbList[] = {32, 64, 96, 128, 160, 192, 256, 320, 384, 512, 640, 768, 896, 1024};
    const int numCfg = sizeof(tpbList)/sizeof(tpbList[0]);
    const int iters = 200;  // langere gemiddelde meting

    // --- Uncoalesced ---
    std::cout<<"=== Part 1: Interleaved (uncoalesced)     R invert ===\n";
    for(int k=0;k<numCfg;++k){
        int tpb = tpbList[k];
        int blocks = (pixels + tpb - 1)/tpb;

        invert_red_interleaved<<<blocks,tpb>>>(dIn,dOut,pixels);
        CUDA_CHECK(cudaDeviceSynchronize());

        cudaEvent_t start, stop;
        CUDA_CHECK(cudaEventCreate(&start));
        CUDA_CHECK(cudaEventCreate(&stop));

        CUDA_CHECK(cudaEventRecord(start));
        for(int it=0; it<iters; ++it)
```

```
            invert_red_interleaved<<<blocks,tpb>>>(dIn,dOut,pixels);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));

        float ms=0; CUDA_CHECK(cudaEventElapsedTime(&ms,start,stop));
        std::cout<<"TPB="<<tpb<<"  blocks="<<blocks
                 <<"  total="<<ms<<" ms  avg="<<(ms/iters)<<" ms\n";

        CUDA_CHECK(cudaEventDestroy(start));
        CUDA_CHECK(cudaEventDestroy(stop));
    }
    CUDA_CHECK(cudaMemcpy(hOutUnc,dOut,total,cudaMemcpyDeviceToHost));
    save_interleaved_as_ppm("./out_part1_uncoalesced.ppm", hOutUnc);

    // --- Coalesced ---
    uint8_t *dR=nullptr,*dG=nullptr,*dB=nullptr,*dRout=nullptr,*dGout=nullptr,*dBout=nullptr;
    CUDA_CHECK(cudaMalloc(&dR,pixels));
    CUDA_CHECK(cudaMalloc(&dG,pixels));
    CUDA_CHECK(cudaMalloc(&dB,pixels));
    CUDA_CHECK(cudaMalloc(&dRout,pixels));
    CUDA_CHECK(cudaMalloc(&dGout,pixels));
    CUDA_CHECK(cudaMalloc(&dBout,pixels));

    { int tpb=256, blocks=(pixels+tpb-1)/tpb;
      deinterleave<<<blocks,tpb>>>(dIn,dR,dG,dB,pixels);
      CUDA_CHECK(cudaDeviceSynchronize());
    }

    std::cout<<"\n=== Part 1: Planar (coalesced)     R invert ===\n";
    for(int k=0;k<numCfg;++k){
        int tpb = tpbList[k];
        int blocks = (pixels + tpb - 1)/tpb;

        invert_red_planar<<<blocks,tpb>>>(dR,dRout,dG,dGout,dB,dBout,pixels);
        CUDA_CHECK(cudaDeviceSynchronize());

        cudaEvent_t start, stop;
        CUDA_CHECK(cudaEventCreate(&start));
        CUDA_CHECK(cudaEventCreate(&stop));

        CUDA_CHECK(cudaEventRecord(start));
        for(int it=0; it<iters; ++it)
            invert_red_planar<<<blocks,tpb>>>(dR,dRout,dG,dGout,dB,dBout,pixels);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));

        float ms=0; CUDA_CHECK(cudaEventElapsedTime(&ms,start,stop));
        std::cout<<"TPB="<<tpb<<"  blocks="<<blocks
                 <<"  total="<<ms<<" ms  avg="<<(ms/iters)<<" ms\n";

        CUDA_CHECK(cudaEventDestroy(start));
        CUDA_CHECK(cudaEventDestroy(stop));
    }

    { int tpb=256, blocks=(pixels+tpb-1)/tpb;
      interleave<<<blocks,tpb>>>(dRout,dGout,dBout,dOut,pixels);
      CUDA_CHECK(cudaDeviceSynchronize());
    }
    CUDA_CHECK(cudaMemcpy(hOutCoa,dOut,total,cudaMemcpyDeviceToHost));
    save_interleaved_as_ppm("./out_part1_coalesced.ppm", hOutCoa);

    // Cleanup
    CUDA_CHECK(cudaFree(dBout)); CUDA_CHECK(cudaFree(dGout)); CUDA_CHECK(cudaFree(dRout));
    CUDA_CHECK(cudaFree(dB)); CUDA_CHECK(cudaFree(dG)); CUDA_CHECK(cudaFree(dR));
    CUDA_CHECK(cudaFree(dOut)); CUDA_CHECK(cudaFree(dIn));
    std::free(hOutCoa); std::free(hOutUnc);

    return 0;
}
```

A.2 Deel 2: Matrix Multiplication (Memory Types)

```cpp
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <iostream>
#include <cmath>
#include <cuda_runtime.h>

#define TILE 16             // tilegrootte voor shared-memory kernel
#define MAX_N 128           // limiet voor constant memory (N x N <= 128 x 128)
__constant__ float B_const[MAX_N * MAX_N];

// ---- Foutcontrole ----
#define CUDA_CHECK(x) do {                                          \
    cudaError_t e = (x);                                            \
    if (e != cudaSuccess) {                                         \
        fprintf(stderr, "CUDA ERROR %s:%d: %s\n",                   \
                __FILE__, __LINE__, cudaGetErrorString(e));         \
        std::exit(EXIT_FAILURE);                                    \
    }                                                               \
} while(0)

// ---- CPU-referentie (controle) ----
void cpu_matmul(const float* A, const float* B, float* C, int N){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            float s = 0.f;
            for(int k = 0; k < N; k++){
                s += A[i * N + k] * B[k * N + j];
            }
            C[i * N + j] = s;
        }
    }
}

// ---- Global-only kernel ----
__global__ void matmul_global(const float* A, const float* B, float* C, int N){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if(row >= N || col >= N) return;

    float sum = 0.f;
    for(int k = 0; k < N; k++){
        sum += A[row * N + k] * B[k * N + col];
    }
    C[row * N + col] = sum;
}

// ---- Shared-memory kernel (tiling met TILE x TILE) ----
// Let op: deze kernel laadt per tile data uit GLOBAL memory in __shared__ arrays As/Bs.
// De gemeten tijd omvat dus zowel global->shared migratie als de eigenlijke
    vermenigvuldiging.
__global__ void matmul_shared(const float* A, const float* B, float* C, int N){
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.f;

    int numTiles = (N + TILE - 1) / TILE;

    for(int t = 0; t < numTiles; t++){
        int aCol = t * TILE + threadIdx.x;
        int bRow = t * TILE + threadIdx.y;

        As[threadIdx.y][threadIdx.x] =
            (row < N && aCol < N) ? A[row * N + aCol] : 0.f;
        Bs[threadIdx.y][threadIdx.x] =
            (bRow < N && col < N) ? B[bRow * N + col] : 0.f;

        __syncthreads();

        #pragma unroll
```

```
        for(int k = 0; k < TILE; k++){
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        }
        __syncthreads();
    }

    if(row < N && col < N){
        C[row * N + col] = sum;
    }
}

// ---- Constant-memory kernel (B in constant memory) ----
__global__ void matmul_const(const float* A, float* C, int N){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if(row >= N || col >= N) return;

    float sum = 0.f;
    for(int k = 0; k < N; k++){
        sum += A[row * N + k] * B_const[k * N + col];
    }
    C[row * N + col] = sum;
}

int main(){
    // Matrixgroottes die we testen
    int sizes[] = {16, 32, 64, 128, 256};
    const int numSizes = sizeof(sizes) / sizeof(sizes[0]);

    // Blockconfiguraties (2D) voor sweep
    int blockSizes[][2] = {
        {8, 8},
        {16, 16},
        {32, 32}
    };
    const int numBlockConfigs = sizeof(blockSizes) / sizeof(blockSizes[0]);

    const int iters = 100;  // aantal kernel-runs per meting (voor gemiddelde)

    for(int idx = 0; idx < numSizes; ++idx){
        int N = sizes[idx];
        size_t bytes = (size_t)N * N * sizeof(float);
        size_t bytesAB = 2 * bytes;   // A en B samen
        size_t bytesC  = bytes;       // C alleen

        std::cout << "\n=========================================\n";
        std::cout << "Matrix size N = " << N << " (N x N)\n";
        std::cout << "=========================================\n";
        std::cout << "Data sizes: A = " << bytes / 1024.0 << " KB, "
                  << "B = " << bytes / 1024.0 << " KB, "
                  << "C = " << bytes / 1024.0 << " KB\n";

        // Host buffers
        std::vector<float> hA(N * N), hB(N * N), hC(N * N), hRef(N * N);
        for(int i = 0; i < N * N; i++){
            hA[i] = (float)((i % 7) + 1);
            hB[i] = (float)((i % 5) + 1);
        }

        // CPU referentie (1x, voor correctheidscheck)
        cpu_matmul(hA.data(), hB.data(), hRef.data(), N);

        // Device allocatie
        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        CUDA_CHECK(cudaMalloc(&dA, bytes));
        CUDA_CHECK(cudaMalloc(&dB, bytes));
        CUDA_CHECK(cudaMalloc(&dC, bytes));

        // ---- Host -> Device (H2D) timing (A + B) ----
        cudaEvent_t h2dS, h2dE;
        CUDA_CHECK(cudaEventCreate(&h2dS));
        CUDA_CHECK(cudaEventCreate(&h2dE));
        CUDA_CHECK(cudaEventRecord(h2dS));
```

```
        CUDA_CHECK(cudaMemcpy(dA, hA.data(), bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(dB, hB.data(), bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaEventRecord(h2dE));
        CUDA_CHECK(cudaEventSynchronize(h2dE));
        float h2d_ms = 0.f;
        CUDA_CHECK(cudaEventElapsedTime(&h2d_ms, h2dS, h2dE));
        CUDA_CHECK(cudaEventDestroy(h2dS));
        CUDA_CHECK(cudaEventDestroy(h2dE));

        std::cout << "H2D (A+B):   " << h2d_ms << " ms  ("
                  << (bytesAB / (h2d_ms / 1000.0)) / 1e9 << " GB/s)\n";

        // ---- Sweep over block sizes ----
        for(int b = 0; b < numBlockConfigs; b++){
            int BX = blockSizes[b][0];
            int BY = blockSizes[b][1];
            int threadsPerBlock = BX * BY;

            dim3 block(BX, BY);
            dim3 grid((N + BX - 1) / BX, (N + BY - 1) / BY);

            std::cout << "\n--- Block size: (" << BX << "x" << BY
                      << "), TPB = " << threadsPerBlock << " ---\n";

            // --- Global-only kernel ---
            {
                cudaEvent_t s1, e1;
                CUDA_CHECK(cudaEventCreate(&s1));
                CUDA_CHECK(cudaEventCreate(&e1));
                CUDA_CHECK(cudaEventRecord(s1));
                for(int i = 0; i < iters; i++){
                    matmul_global<<<grid, block>>>(dA, dB, dC, N);
                }
                CUDA_CHECK(cudaEventRecord(e1));
                CUDA_CHECK(cudaEventSynchronize(e1));
                float t1 = 0.f;
                CUDA_CHECK(cudaEventElapsedTime(&t1, s1, e1));
                CUDA_CHECK(cudaEventDestroy(s1));
                CUDA_CHECK(cudaEventDestroy(e1));

                std::cout << "Global-only: " << t1
                          << " ms total (" << t1 / iters << " ms avg)\n";
            }

            // --- Shared-memory kernel ---
            if (BX == TILE && BY == TILE) {
                cudaEvent_t s2, e2;
                CUDA_CHECK(cudaEventCreate(&s2));
                CUDA_CHECK(cudaEventCreate(&e2));
                CUDA_CHECK(cudaEventRecord(s2));
                for(int i = 0; i < iters; i++){
                    matmul_shared<<<grid, block>>>(dA, dB, dC, N);
                }
                CUDA_CHECK(cudaEventRecord(e2));
                CUDA_CHECK(cudaEventSynchronize(e2));
                float t2 = 0.f;
                CUDA_CHECK(cudaEventElapsedTime(&t2, s2, e2));
                CUDA_CHECK(cudaEventDestroy(s2));
                CUDA_CHECK(cudaEventDestroy(e2));

                std::cout << "Shared:      " << t2
                          << " ms total (" << t2 / iters << " ms avg)\n";
            } else {
                std::cout << "Shared:      skipped (blockDim != TILE="
                          << TILE << ")\n";
            }

            // --- Constant-memory kernel ---
            if (N <= MAX_N) {
                CUDA_CHECK(cudaMemcpyToSymbol(B_const, hB.data(), bytes));
                cudaEvent_t s3, e3;
                CUDA_CHECK(cudaEventCreate(&s3));
                CUDA_CHECK(cudaEventCreate(&e3));
```

```
                CUDA_CHECK(cudaEventRecord(s3));
                for(int i = 0; i < iters; i++){
                    matmul_const<<<grid, block>>>(dA, dC, N);
                }
                CUDA_CHECK(cudaEventRecord(e3));
                CUDA_CHECK(cudaEventSynchronize(e3));
                float t3 = 0.f;
                CUDA_CHECK(cudaEventElapsedTime(&t3, s3, e3));
                CUDA_CHECK(cudaEventDestroy(s3));
                CUDA_CHECK(cudaEventDestroy(e3));

                std::cout << "Constant(B): " << t3
                          << " ms total (" << t3 / iters << " ms avg)\n";
            } else {
                std::cout << "Constant(B): skipped (N > MAX_N=" << MAX_N << ")\n";
            }
        }

        // ---- Device -> Host (D2H) timing (C) ----
        cudaEvent_t d2hS, d2hE;
        CUDA_CHECK(cudaEventCreate(&d2hS));
        CUDA_CHECK(cudaEventCreate(&d2hE));
        CUDA_CHECK(cudaEventRecord(d2hS));
        CUDA_CHECK(cudaMemcpy(hC.data(), dC, bytes, cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaEventRecord(d2hE));
        CUDA_CHECK(cudaEventSynchronize(d2hE));
        float d2h_ms = 0.f;
        CUDA_CHECK(cudaEventElapsedTime(&d2h_ms, d2hS, d2hE));
        CUDA_CHECK(cudaEventDestroy(d2hS));
        CUDA_CHECK(cudaEventDestroy(d2hE));

        std::cout << "\nD2H (C):     " << d2h_ms << " ms  ("
                  << (bytesC / (d2h_ms / 1000.0)) / 1e9 << " GB/s)\n";

        // ---- Correctheidscheck ----
        double mae = 0.0;
        for(int i = 0; i < N * N; i++){
            mae += std::fabs(hC[i] - hRef[i]);
        }
        std::cout << "Mean abs error vs CPU: " << mae / (N * N) << "\n";

        // Cleanup
        CUDA_CHECK(cudaFree(dA));
        CUDA_CHECK(cudaFree(dB));
        CUDA_CHECK(cudaFree(dC));
    }

    return 0;
}
```

A.3 Deel 3: Bandwidth Measurement

```
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <cuda_runtime.h>

#define CUDA_CHECK(x) do {                                          \
    cudaError_t e = (x);                                            \
    if (e != cudaSuccess) {                                         \
        fprintf(stderr, "CUDA ERROR %s:%d: %s\n",                   \
                __FILE__, __LINE__, cudaGetErrorString(e));         \
        std::exit(EXIT_FAILURE);                                    \
    }                                                               \
} while(0)

// Simpele kernel: leest global geheugen in shared memory
// Dit simuleert "GPU global -> GPU shared" migratie.
__global__ void g2s_kernel(const float* __restrict__ dIn, int N){
    extern __shared__ float s[];   // dynamisch shared memory
    int tid   = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    int stride= blockDim.x * gridDim.x;

    for (int i = tid; i < N; i += stride) {
        // elke thread laad   n   element in shared
        s[threadIdx.x] = dIn[i];
        __syncthreads();
        // we doen niets met s[], we willen enkel de load-tijd meten
    }
}

int main(){
    // Grootte van de array (bijvoorbeeld ~1M floats)
    const int N = 1 << 20; // 1.048.576
    const size_t bytes = N * sizeof(float);
    const int iters = 100;

    std::cout << "N = " << N << " elements (" << bytes / (1024.0 * 1024.0)
              << " MB)\n";

    // Host alloc + init
    float* hIn = (float*)std::malloc(bytes);
    if (!hIn) { std::cerr << "malloc failed\n"; return 1; }
    for (int i = 0; i < N; ++i) hIn[i] = (float)(i % 100);

    // Device alloc
    float* dIn = nullptr;
    CUDA_CHECK(cudaMalloc(&dIn, bytes));

    // ======= H2D meting (CPU -> GPU global) =======
    cudaEvent_t h2dStart, h2dStop;
    CUDA_CHECK(cudaEventCreate(&h2dStart));
    CUDA_CHECK(cudaEventCreate(&h2dStop));

    CUDA_CHECK(cudaEventRecord(h2dStart));
    for (int it = 0; it < iters; ++it) {
        CUDA_CHECK(cudaMemcpy(dIn, hIn, bytes, cudaMemcpyHostToDevice));
    }
    CUDA_CHECK(cudaEventRecord(h2dStop));
    CUDA_CHECK(cudaEventSynchronize(h2dStop));

    float h2d_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&h2d_ms, h2dStart, h2dStop));

    double h2d_avg_ms = h2d_ms / iters;
    double h2d_bw_GBps = (bytes / (h2d_avg_ms / 1000.0)) / 1e9;

    std::cout << "\n=== H2D (Host -> Device global) ===\n";
    std::cout << "Total time over " << iters << " copies: " << h2d_ms << " ms\n";
    std::cout << "Avg per copy: " << h2d_avg_ms << " ms  ("
              << h2d_bw_GBps << " GB/s)\n";

    CUDA_CHECK(cudaEventDestroy(h2dStart));
    CUDA_CHECK(cudaEventDestroy(h2dStop));

    // ======= G2S meting (GPU global -> GPU shared) =======
    // Simpele configuratie
    int tpb = 256;
    int blocks = (N + tpb - 1) / tpb;

    cudaEvent_t g2sStart, g2sStop;
    CUDA_CHECK(cudaEventCreate(&g2sStart));
    CUDA_CHECK(cudaEventCreate(&g2sStop));

    // shared memory size per block = tpb * sizeof(float)
    size_t shmemBytes = tpb * sizeof(float);

    // warm-up
    g2s_kernel<<<blocks, tpb, shmemBytes>>>(dIn, N);
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaEventRecord(g2sStart));
    for (int it = 0; it < iters; ++it) {
        g2s_kernel<<<blocks, tpb, shmemBytes>>>(dIn, N);
```

```
    }
    CUDA_CHECK(cudaEventRecord(g2sStop));
    CUDA_CHECK(cudaEventSynchronize(g2sStop));

    float g2s_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&g2s_ms, g2sStart, g2sStop));

    double g2s_avg_ms = g2s_ms / iters;

    // Hoeveel data wordt ongeveer gelezen? Elke element wordt 1x uit global gelezen.
    double totalBytesRead = (double)bytes; // per kernel-run
    double g2s_bw_GBps = (totalBytesRead / (g2s_avg_ms / 1000.0)) / 1e9;

    std::cout << "\n=== G2S (Device global -> Shared) ===\n";
    std::cout << "Total time over " << iters << " kernel runs: " << g2s_ms << " ms\n";
    std::cout << "Avg per run: " << g2s_avg_ms << " ms  ("
              << g2s_bw_GBps << " GB/s, approx)\n";

    CUDA_CHECK(cudaEventDestroy(g2sStart));
    CUDA_CHECK(cudaEventDestroy(g2sStop));

    // Cleanup
    CUDA_CHECK(cudaFree(dIn));
    std::free(hIn);

    return 0;
}
```